



CS-317



Operating Systems

LECTURE 12



Concurrency

BOOK 1 CHAPTER 5

In the last lecture...



- ▶ We were introduced to the concept of concurrency.
- ▶ We realized that sometimes concurrency is desirable and sometimes it is problematic.
- ▶ We realized that concurrency related problems always exist for shared modifiable data.
- ▶ We realized what race condition is.
- ▶ We were introduced to the types of process interaction.



Process Interaction

PROBLEMS ARISING OUT OF CONCURRENCY

How to classify process interaction?

► We can classify the ways in which processes interact based on the degree to which they are aware of each other's existence.

► Processes unaware of each other,

► Processes indirectly aware of each other,

► Processes directly aware of each other.



Problems of competing processes

- ▶ In the case of competing processes, we face three control problems:
 - ▶ Mutual exclusion, 
 - ▶ Deadlock, and
 - ▶ Starvation.
- ▶ The last two problems are a consequence of the first.

Mutual exclusion

- ▶ When two or more processes require access to a **non-sharable resource**, one of them is given access, while all the others are made to wait. As long as a process uses the resource, access to the resource is not granted to anyone else. This discipline is referred to as Mutual Exclusion (ME).
- ▶ The resource on which this discipline is enforced is called a **critical resource**.
the resource that is wanted
- ▶ The lines of code that use a critical resource are collectively called the **critical section** of the process.
- ▶ It is important that only one program at a time be allowed in its critical section.

Mutual exclusion

- ▶ There are n processes.
- ▶ They share a resource R_o .
- ▶ The **entercritical** function lets only one of the processes calling it to move to next line of code after it.
- ▶ When a process calls **exitcritical** function, a signal is sent to **entercritical**, to let one process move to the next line of code.

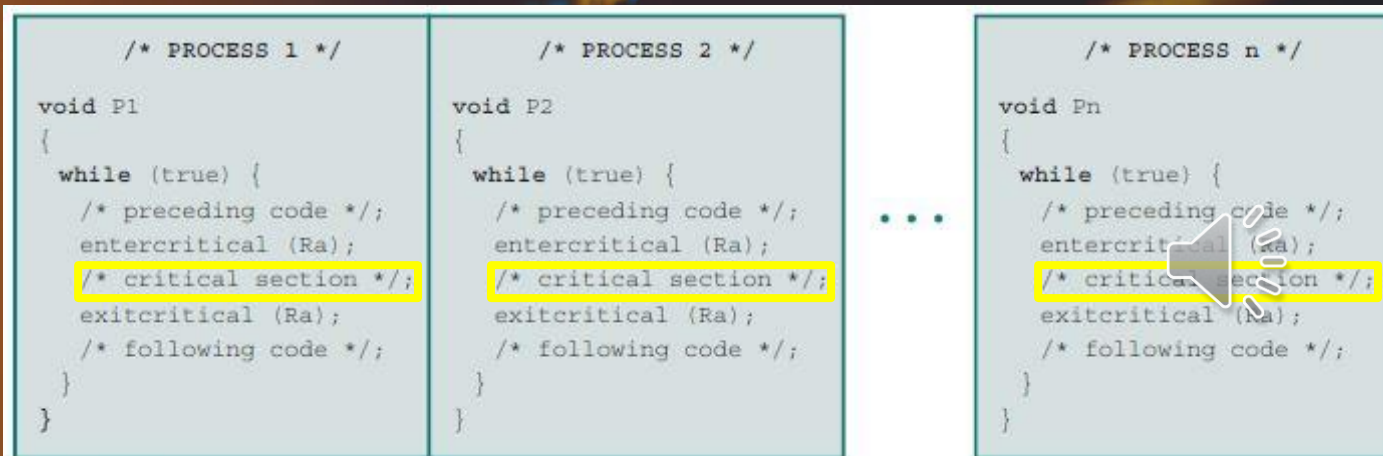


Figure 5.1 Illustration of Mutual Exclusion

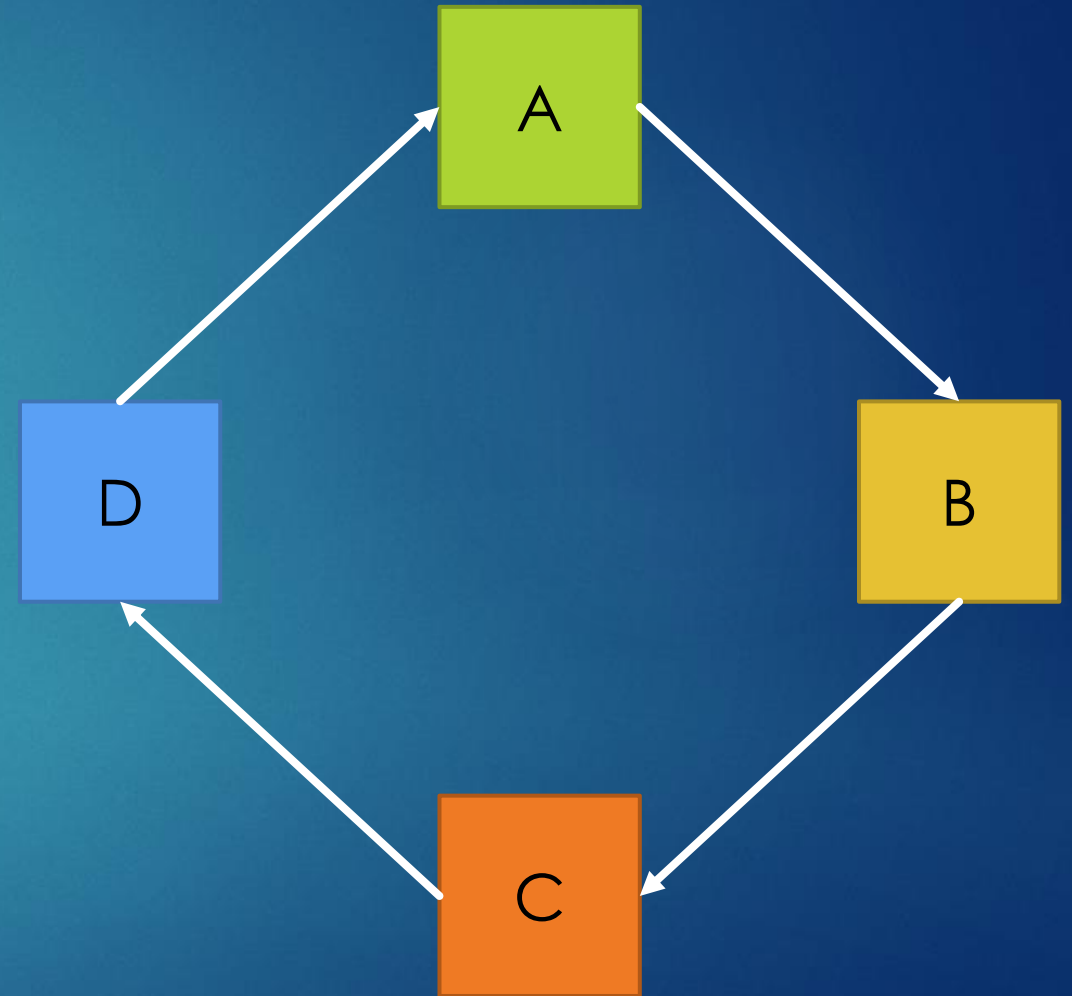
Deadlock

- ▶ Consider two processes, P1 and P2, and two resources, R1 and R2.
- ▶ Suppose that each process needs access to both resources to perform its function.
- ▶ A process leaves an allocated resource only when it has finished its work.
- ▶ The OS assigns R1 to P2, and R2 to P1.
- ▶ Each process is waiting for one of the two resources.
- ▶ Neither will release the resource that it already owns until it has acquired the other resource and performed its task.
- ▶ The two processes are deadlocked.

More about deadlock

► A deadlock is a circle of want.

► It can also be called a circle of wait.



Starvation

1. Suppose that three processes P1, P2 and P3 each require periodic access to resource R.
2. The OS grants R to P1, and both P2 and P3 are waiting.
3. When P1 exits its critical section, it relinquishes R.
4. Either P2 or P3 should be allowed access to R.
5. The OS grants access to P3. Before P3 comes out of critical section, P1 again asks for access to R.
6. The OS grants access to P1 after P3 has finished. Before P1 comes out of critical section, P3 again asks for access to R.
7. The system keeps alternating between points 5 and 6.
8. P2 may be denied access to the resource indefinitely. P2 is starving.



Pause the video!

Write your answer in a word file. Save it as <my_roll_no>_lecture_12. If your roll no is 500, the file will be named 500_lecture_12. Do not add <> signs to the filename.

Starvation

A process keeps waiting for a long time (possibly forever).

Happens due to unfair resource allocation

Other processes keep getting priority

-biasness included

-no loop

-includes activity

-may resolve after pre long period

Task# 1: What is the difference between starvation and deadlock?



past paper

Deadlock = everyone stuck

Starvation = someone ignored

Deadlock

A situation where processes are stuck forever

Each process is waiting for a resource held by another

Forms a circular wait

-includes loop

-no activity

-will not resolve

Processes indirectly aware of each other

the shared users are unknown but is known that there are other users

- ▶ These are processes that are not necessarily aware of each other by their respective process IDs but that share access to some object.
- ▶ Example: multiple processes having access to shared variables or to shared files or databases, processes reading from and writing to an I/O buffer.
- ▶ Such processes exhibit cooperation by sharing the common object.

Cooperation by sharing

- ▶ Processes may use and update the shared data without reference to other processes but know that other processes may have access to the same data.
- ▶ The control mechanisms must ensure the integrity of the shared data.

Problems of processes cooperating by sharing

- ▶ In the case of processes cooperating by sharing, we face four control problems:
 - ▶ Mutual exclusion,
 - ▶ Deadlock,
 - ▶ Starvation, and
 - ▶ Data coherence.
- ▶ Data can be used two ways: it can be read or written.
- ▶ Mutual exclusion only needs to be enforced when the shared resource is being written to.



Data coherence



parallel execution

- ▶ Data that is **coherent** is data that is the same in all places where it is stored.
- ▶ In an application, various data items are updated.
- ▶ **a** and **b** are to be maintained in the relationship **a = b**.
- ▶ Any program that updates one value must also update the other.
- ▶ Two processes **P1** and **P2** contain code shown in box 1.
- ▶ **P1** and **P2** execute instructions according to the sequence in box 2.
- ▶ **a = b = 1** before execution shown in box 2.

BOX 1

P1:

a = **a** + 1

b = **b** + 1

P2:

b = 2 × **b**

a = 2 × **a**

BOX 2

a = **a** + 1

b = 2 × **b**

b = **b** + 1

a = 2 × **a**

a 1 **b** 1

Data coherence

- ▶ Data that is **coherent** is data that is the same in all places where it is stored.
- ▶ In an application, various data items are updated.
- ▶ **a** and **b** are to be maintained in the relationship **a = b**.
- ▶ Any program that updates one value must also update the other.
- ▶ Two processes **P1** and **P2** contain code shown in box 1.
- ▶ **P1** and **P2** execute instructions according to the sequence in box 2.
- ▶ **a = b = 1** before execution shown in box 2.

BOX 1

P1:

$a = a + 1$

$b = b + 1$

P2:

$b = 2 \times b$

$a = 2 \times a$

BOX 2

$a = a + 1$

$b = 2 \times b$

$b = b + 1$

$a = 2 \times a$

a 2 **b** 1

Data coherence

- ▶ Data that is **coherent** is data that is the same in all places where it is stored.
- ▶ In an application, various data items are updated.
- ▶ **a** and **b** are to be maintained in the relationship **a = b**.
- ▶ Any program that updates one value must also update the other.
- ▶ Two processes **P1** and **P2** contain code shown in box 1.
- ▶ **P1** and **P2** execute instructions according to the sequence in box 2.
- ▶ **a = b = 1** before execution shown in box 2.

BOX 1

P1:

a = **a** + 1

b = **b** + 1

P2:

b = 2 × **b**

a = 2 × **a**

BOX 2

a = **a** + 1

b = 2 × **b**

b = **b** + 1

a = 2 × **a**

a 2 **b** 2

Data coherence

- ▶ Data that is **coherent** is data that is the same in all places where it is stored.
- ▶ In an application, various data items are updated.
- ▶ **a** and **b** are to be maintained in the relationship **a = b**.
- ▶ Any program that updates one value must also update the other.
- ▶ Two processes **P1** and **P2** contain code shown in box 1.
- ▶ **P1** and **P2** execute instructions according to the sequence in box 2.
- ▶ **a = b = 1** before execution shown in box 2.

BOX 1

P1:

a = **a** + 1

b = **b** + 1

P2:

b = 2 × **b**

a = 2 × **a**

BOX 2

a = **a** + 1

b = 2 × **b**

b = **b** + 1

a = 2 × **a**

a 2 **b** 3

Data coherence

- ▶ Data that is **coherent** is data that is the same in all places where it is stored.
- ▶ In an application, various data items are updated.
- ▶ **a** and **b** are to be maintained in the relationship **a = b**.
- ▶ Any program that updates one value must also update the other.
- ▶ Two processes **P1** and **P2** contain code shown in box 1.
- ▶ **P1** and **P2** execute instructions according to the sequence in box 2.
- ▶ **a = b = 1** before execution shown in box 2.

BOX 1

P1:

a = **a** + 1

b = **b** + 1

P2:

b = 2 × **b**

a = 2 × **a**

BOX 2

a = **a** + 1

b = 2 × **b**

b = **b** + 1

a = 2 × **a**

a 4 **b** 3

Data coherence

- ▶ Data that is **coherent** is data that is the same in all places where it is stored.
- ▶ In an application, various data items are updated.
- ▶ **a** and **b** are to be maintained in the relationship **a = b**.
- ▶ Any program that updates one value must also update the other.
- ▶ Two processes **P1** and **P2** contain code shown in box 1.
- ▶ **P1** and **P2** execute instructions according to the sequence in box 2.
- ▶ **a = b = 1** before execution shown in box 2.
- ▶ The two variables have lost coherence.

BOX 1

P1:

a = **a** + 1

b = **b** + 1

P2:

b = 2 × **b**

a = 2 × **a**

BOX 2

a = **a** + 1

b = 2 × **b**

b = **b** + 1

a = 2 × **a**

a 4 **b** 3

Solution

olution for data coherence issue is to enforce mutual exclusion

```
/*PROCESS 1*/
void P1
{
    while(true) {
        /*preceding code*/;
        entercritical(xyz);
        a = a + 1;
        b = b + 1;
        exitcritical(xyz);
        /*following code*/;
    }
}
```



```
/*PROCESS 2*/
void P2
{
    while(true) {
        /*preceding code*/;
        entercritical(xyz);
        b = 2 × b;
        a = 2 × a;
        exitcritical(xyz);
        /*following code*/;
    }
}
```

Processes directly aware of each other

- ▶ These are processes that can communicate with each other by process ID and that are designed to work jointly on some activity.
- ▶ Example: A process spawns a child. The parent process asks the child process to do a task. Both processes know each other and communicate with each other.
- ▶ Such processes exhibit cooperation by communication.

Cooperation by communication

- ▶ When processes cooperate by communication, they participate in a common effort.
- ▶ The communication provides a way to **synchronize**, or coordinate, or order the various activities.
- ▶ Communication consists of messages of some sort.
- ▶ Functions for sending and receiving messages are provided by programming language or the OS kernel.

Problems of processes cooperating by communication

- ▶ In the case of processes cooperating by communication, we face two control problems:
 - ▶ Deadlock, and
 - ▶ Starvation.
- ▶ Because nothing is shared between processes in the act of passing messages, mutual exclusion is not a control requirement here.



Starvation



- ▶ P1 periodically sends a message to either P2 or P3.
- ▶ When any one of P2 and P3 receives a message from P1, it executes some code.
- ▶ As long as P2 and P3 do not receive a message from P1, they wait.
- ▶ P1 send a message to P2, P2 executes its code.
- ▶ P1 again selects P2 and sends a message to P2. P2 executes its code again.
- ▶ P1 again selects P2 and sends a message to P2. P2 executes its code again.
- ▶ P3 is waiting for a message from P1. There is no deadlock, because P1 remains active, but P3 is starved.

Bringing it all together...

Table 5.2 Process Interaction

Degree of Awareness	Relationship	Influence that One Process Has on the Other	Potential Control Problems
Processes unaware of each other	Competition	• Results of one process independent of the action of others • Timing of process may be affected	• Mutual exclusion • Deadlock (renewable resource) • Starvation echo issue me
Processes indirectly aware of each other (e.g., shared object)	Cooperation by sharing	• Results of one process may depend on information obtained from others • Timing of process may be affected	• Mutual exclusion • Deadlock (renewable resource) • Starvation • Data coherence
Processes directly aware of each other (have communication primitives available to them)	Cooperation by communication	• Results of one process may depend on information obtained from others • Timing of process may be affected	• Deadlock (consumable resource) • Starvation

- Conditions are not always as clear-cut as suggested in this table.
- Several processes may exhibit aspects of both competition and cooperation.

An example of processes that are unaware of each other is:

A music player app playing songs
A web browser loading websites

in os, both doesnot know each other

Requirements of ME

1. Mutual exclusion must be enforced: only one process at a time is allowed into its critical section, among all processes that have critical sections for the same resource or shared object.
2. A process that halts in its noncritical section must do so without interfering with other processes.
3. It must not be possible for a process requiring access to a critical section to be delayed indefinitely: no deadlock or starvation.
4. When no process is in a critical section, any process that requests entry to its critical section must be permitted to enter without delay.
5. No assumptions are made about relative process speeds or number of processors.
6. A process remains in its critical section for a finite time only.

Pause the video!

Write your answer in a word file previously saved as

<my_roll_no>_lecture_12.

Task# 2: What is the difference between the 3rd and 6th requirement of mutual exclusion?

How to satisfy the requirements of ME?

- ▶ Leave the responsibility with the processes that wish to execute concurrently.
 - ▶ We can refer to this as the **software approach**.
 - ▶ This approach is prone to high processing overhead and bugs.
- ▶ Make use of special purpose machine instructions.
 - ▶ We can refer to this as the **hardware approach**.
 - ▶ They reduce overheads.
 - ▶ They are unattractive as a general-purpose solution.
- ▶ Provide some level of support within the OS or a programming language.





Mutual Exclusion

HARDWARE SUPPORT

os kernel does this work

Interrupt disabling

```
while (true)
{
/*disable Interrupts * / ;
/* critical section * / ;
/* enable interrupts * / ;
/* remainder * / ;
}
```

- ▶ In a uniprocessor system, concurrent processes cannot have overlapped execution; they can only be interleaved.
- ▶ A process will continue to run until it invokes an OS service or until it is interrupted.
- ▶ To guarantee mutual exclusion, it is sufficient to prevent a process from being interrupted.
- ▶ Primitives are defined by the OS kernel for disabling and enabling interrupts.
- ▶ A process can enforce mutual exclusion by disabling interrupts before entering critical section and enabling them after exiting from critical section.

Primitives

- ▶ A primitive is the **smallest 'unit of processing'** available to a programmer of a given machine.
- ▶ Primitives are **atomic elements** 
- ▶ An atomic operation is a function or a sequence of instructions which cannot be interrupted.
- ▶ **Any other process cannot access the values of variables of an atomic operation during the operation.**
- ▶ We say an **atomic operation is indivisible.**
- ▶ Atomic operations either execute completely or do not execute at all.

Analysis of interrupt disabling

- ▶ Because the critical section cannot be interrupted, **mutual exclusion is guaranteed.**
- ▶ The efficiency of execution could be noticeably degraded because the processor is limited in its ability to interleave processes. since, processor cannot be interrupted to be given to another process : interleaving
- ▶ **This approach will not work in a multiprocessor architecture.**

Special machine instructions



interrupt disabling can work in a uni processor but for multiprocessor we need special machine instructions

- ▶ In a symmetric multiprocessor, several processors share access to a common main memory.
- ▶ At the hardware level, access to a memory location excludes any other access to that same location.
- ▶ Designers have proposed several machine instructions that carry out two actions atomically, such as reading and writing or reading and testing, of a single memory location with one instruction fetch cycle.
- ▶ During execution of the instruction, access to the memory location is blocked for any other instruction referencing that location.

Special machine instructions

- ▶ There are two machine instructions which deserve a mention here:

- ▶ Compare and swap 

- ▶ Exchange

Compare & swap instruction

```
int compare_and_swap (int *word, int testval, int newval)
{
    int oldval ;
    oldval = *word;
    if (oldval == testval) *word = newval;
    return oldval;
}
```

ATOMIC



- ▶ It is also called compare and exchange instruction.
- ▶ It checks a memory location `*word` against a test value `testval`.
- ▶ If the memory location's current value is equal to `testval`, it is replaced with `newval`; otherwise it is left unchanged.
- ▶ The old memory value is always returned.
- ▶ If the returned value is equal to `testval`, the memory location has been updated.

Compare & swap – Boolean version

```
bool compare_and_swap (int *word, int testval, int newval)
{
    int oldval ;
    oldval = *word;
    if (oldval == testval) {
        *word = newval;
        return TRUE;
    }
    return FALSE;
}
```



- ▶ This version returns a Boolean value: true if the swap occurred; false otherwise.
- ▶ Some version of this instruction is available on nearly all processor families (x86, IA64, spare, IBM z series, etc.), and most operating systems use this instruction for support of concurrency.

Lessons learned



- ▶ We looked in detail at all the ways processes interact.
- ▶ We discussed examples of deadlock, starvation and data coherence.
- ▶ We realized the requirements of ME.
- ▶ We delved into the hardware approach to implement ME.

Things to do



Read the book.



I will ask you to submit the word file previously saved as <my_roll_no>_lecture_12 in Google Classroom.



Write your questions on a piece of paper and keep it in front of you during the online session.



If you have suggestions relevant to content or quality of this lecture, email them to me.